# WorkHub Backend API Guide for Frontend

## 1. Overview

Tài liệu này mô tả các API backend hiện có của WorkHub để frontend tích hợp các luồng:

- Xác thực người dùng.
- Quản lý thư mục và tài liệu.
- Quản lý task, assignee, checklist và timeline hoạt động.
- Thông báo trong app.
- Admin dashboard, khóa/mở khóa user và audit log.

Base URL giả định:

```
http://localhost:5000/api
```

Backend dùng JSON cho phần lớn API, ngoại trừ upload tài liệu dùng `multipart/form-data` và download/preview trả về file stream/blob.

Response thành công không có wrapper chung cố định. Tùy endpoint có thể trả object trực tiếp, `{ content, totalElements }`, `{ message }`, hoặc file stream.

Error format phổ biến:

```
{
  "message": "Server error, please try again"
}
```

Auth dùng JWT Bearer:

```
Authorization: Bearer <token>
```

Security notes cho frontend:

- Không dùng, không hiển thị, không lưu `storageKey`.
- Không tự build Cloudflare R2 URL. File phải đi qua backend download/preview/share endpoint.
- User có `status = "locked"` hoặc `"disabled"` không thể login và không thể gọi protected APIs.
- `totalElements` ở task list và folder document list được thiết kế để chỉ tính resource user hiện tại có quyền đọc.
- Backend là source of truth cho permission. Frontend có thể ẩn nút, nhưng vẫn phải xử lý `403`.

## 2. Common Conventions

### Headers

Protected endpoints:

```
Authorization: Bearer {{token}}
```

JSON endpoints:

```
Content-Type: application/json
```

Upload tài liệu:

```
Content-Type: multipart/form-data
Authorization: Bearer {{token}}
```

### Pagination

Các tham số đang dùng trong code:

| Khu vực | Params |
|---|---|
| Tasks | `page`, `size` |
| Folder documents | `page`, `size` |
| Notifications | `page`, `limit` hoặc `size` |
| Admin activity logs | `page`, `limit` hoặc `size` |

Giới hạn tối đa:

- Tasks: `size` tối đa `100`.
- Folder documents: `size` tối đa `100`.
- Notifications: `limit` tối đa `100`.
- Admin activity logs: `limit` tối đa `100`.

### Date Format

Nên gửi ISO 8601 string:

```
{
  "endAt": "2026-05-01T10:00:00.000Z"
}
```

### Common Status Codes

| Code | Ý nghĩa |
|---|---|
| 200 | Thành công |
| 201 | Tạo mới thành công |
| 202 | Upload idempotency đang xử lý hoặc resource chưa active |
| 204 | Xóa/soft delete thành công, không có body |
| 400 | Request sai, thiếu field, validation lỗi |
| 401 | Chưa đăng nhập/token lỗi |
| 403 | Không đủ quyền hoặc account bị khóa/vô hiệu |
| 404 | Không tìm thấy hoặc không được reveal resource |
| 409 | Duplicate/resource conflict |
| 410 | Share link hết hạn hoặc hết lượt tải |
| 413 | File quá lớn |
| 501 | Endpoint reset password admin là placeholder |

### Soft Delete

- Task: `DELETE /tasks/:id` set `deletedAt`, list mặc định loại task đã xóa.
- Folder: `DELETE /folders/:id` set `deletedAt`.
- Document: `DELETE /documents/:id` set `status = "deleted"` và `deletedAt`.
- Notification: `DELETE /notifications/:id` set `deletedAt`.

## 3. Auth

### Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| POST | `/auth/register` | No | Đăng ký user local, gửi OTP email |
| POST | `/auth/login` | No | Login email/password |
| POST | `/auth/google` | No | Login Google |
| POST | `/auth/verify-email` | No | Verify email bằng OTP |
| POST | `/auth/resend-otp` | No | Gửi lại OTP |
| POST | `/auth/forgot-password` | No | Gửi reset password email |
| POST | `/auth/reset-password/:token` | No | Reset password bằng token |
| GET | `/auth/me` | Yes | Lấy profile user hiện tại |

Không có endpoint logout/refresh token trong code hiện tại. Frontend logout bằng cách xóa token ở client.

### `POST /auth/login`

Request:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response `200`:

```
{
  "_id": "userId",
  "fullName": "Nguyen Van A",
  "email": "user@example.com",
  "role": "user",
  "avatar": "",
  "token": "jwt-token"
}
```

Error cases:

- `400`: thiếu email/password.
- `401`: sai email/password hoặc account Google không có password local.
- `403`: chưa verify email, hoặc user `locked`/`disabled`.

Frontend notes:

- Lưu `token` để gọi protected APIs.
- Khi gặp `403` với message account not active, hiển thị thông báo liên hệ admin.

### `POST /auth/google`

Request body theo code hiện tại:

```
{
  "email": "user@example.com",
  "name": "Nguyen Van A",
  "picture": "https://...",
  "sub": "google-user-id"
}
```

Response giống login:

```
{
  "_id": "userId",
```

```
  "fullName": "Nguyen Van A",
  "email": "user@example.com",
  "role": "user",
  "avatar": "https://...",
  "token": "jwt-token"
}
```

Error cases:

- `400`: thiếu `email` hoặc `sub`.
- `403`: user đã tồn tại nhưng `locked`/`disabled`.
- `500`: xác thực Google thất bại.

Needs confirmation from implementation:

- Backend hiện tại nhận thông tin Google từ frontend nhưng không thấy bước verify Google token bằng `google-auth-library` trong presenter.

### `GET /auth/me`

Auth: required.

Response `200`:

```
{
  "_id": "userId",
  "fullName": "Nguyen Van A",
  "email": "user@example.com",
  "role": "user",
  "avatar": ""
}
```

Error cases:

- `401`: token thiếu/sai.
- `403`: user `locked`/`disabled`.
- `404`: user không tồn tại.

## Other Auth Endpoints

### `POST /auth/register`

Request:

```
{
  "fullName": "Nguyen Van A",
  "email": "user@example.com",
```

```
  "password": "password123"
}
```

Response `201`:

```
{
  "message": "Registration successful! A verification code has been sent to your email.",
  "email": "user@example.com"
}
```

#### `POST /auth/verify-email`

Request:

```
{
  "email": "user@example.com",
  "otp": "123456"
}
```

Response `200` gồm user info và `token`.

#### `POST /auth/resend-otp`

Request:

```
{
  "email": "user@example.com"
}
```

Response:

```
{
  "message": "A new verification code has been sent to your email."
}
```

#### `POST /auth/forgot-password`

Request:

```
{
  "email": "user@example.com"
```

```
}
```

Response luôn chung chung để tránh leak email tồn tại:

```
{
  "message": "If an account with that email exists, a password reset link has been sent."
}
```

`POST /auth/reset-password/:token`

Request:

```
{
  "password": "new-password"
}
```

Response:

```
{
  "message": "Password reset successful! You can now log in with your new password."
}
```

## 4. Folders and Documents

### Folder Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| `GET` | `/folders` | Yes | Danh sách thư mục theo `parentId`, `departmentId` |
| `POST` | `/folders` | Yes | Tạo thư mục |
| `GET` | `/folders/:id` | Yes | Chi tiết thư mục và children |
| `PUT` | `/folders/:id` | Yes | Cập nhật thư mục |
| `DELETE` | `/folders/:id` | Yes | Soft delete thư mục |
| `GET` | `/folders/:id/documents` | Yes | Danh sách document trong thư mục |
| `POST` | `/folders/:id/documents` | Yes | Upload document vào thư mục |

### `GET /folders`

Query:

| Param | Required | Ghi chú |
|---|---|---|
| `parentId` | No | Nếu không gửi, backend lọc `parentId = null` |
| `departmentId` | No | Lọc theo department |

Response:

```
[
  {
    "_id": "folderId",
    "name": "Contracts",
    "parentId": null,
    "departmentId": null,
    "ownerId": "userId",
    "createdBy": "userId",
    "permissions": {
      "visibility": "private",
      "users": []
    },
    "deletedAt": null,
    "createdAt": "2026-04-27T00:00:00.000Z",
    "updatedAt": "2026-04-27T00:00:00.000Z"
  }
]
```

### `POST /folders`

Request:

```
{
  "name": "Contracts",
  "parentId": null,
  "departmentId": null,
  "permissions": {
    "visibility": "private",
    "users": []
  }
}
```

Response `201`: folder object.

Permission:

- Root folder: user đăng nhập có thể tạo.
- Folder con: phải đọc được parent và có quyền editor/owner/admin trên parent.

### `GET /folders/:id`

Response:

```
{
  "folder": {
    "_id": "folderId",
    "name": "Contracts"
  },
  "children": [
    {
      "type": "folder",
      "item": {
        "_id": "childFolderId",
        "name": "2026"
      }
    },
    {
      "type": "document",
      "item": {
        "_id": "documentId",
        "name": "report.pdf",
        "status": "active"
      }
    }
  ]
}
```

### `PUT /folders/:id`

Request:

```
{
  "name": "New folder name",
  "permissions": {
    "visibility": "custom",
    "users": [
      {
        "userId": "userId",
        "role": "viewer"
      }
    ]
  }
}
```

Response `200`: updated folder object.

### `DELETE /folders/:id`

Response `204`. Backend set `deletedAt`.

### `GET /folders/:id/documents`

Query:

| Param | Default | Max |
|---|---|---|
| `page` | 1 | - |
| `size` | 20 | 100 |

Response:

```
{
  "content": [
    {
      "_id": "documentId",
      "name": "report.pdf",
      "description": "Quarter report",
      "status": "active",
      "folderId": "folderId",
      "currentVersionId": "versionId",
      "createdAt": "2026-04-27T00:00:00.000Z",
      "updatedAt": "2026-04-27T00:00:00.000Z"
    }
  ],
  "totalElements": 1
}
```

`totalElements` chỉ tính document user hiện tại có quyền đọc.

Frontend không nên expect hoặc dùng `storageKey` trong list.

### Upload Document: `POST /folders/:id/documents`

Auth: required.

Content type: `multipart/form-data`.

Fields:

| Field | Type | Required |
|---|---|---|
| `file` | File | Yes |
| `description` | string | No |

Optional header:

```
Idempotency-Key: unique-client-upload-id
```

Max size:

- `MAX_UPLOAD_SIZE_MB` env, default `50MB`.

Allowed extensions:

```
.pdf, .docx, .xlsx, .pptx, .png, .jpg, .jpeg, .txt
```

Blocked extensions:

```
.html, .htm, .svg, .js, .mjs, .exe, .sh, .bat, .cmd, .ps1
```

Backend kiểm tra extension và magic bytes/file signature cho các loại không phải `.txt`.

Response `201`:

```
{
  "_id": "documentId",
  "name": "report.pdf",
  "description": "Quarter report",
  "folderId": "folderId",
  "status": "active",
  "currentVersionId": "versionId",
  "versionCounter": 1
}
```

Idempotency behavior:

- Nếu cùng `createdBy + Idempotency-Key` đã tồn tại:
  - `200` nếu document active.
  - `202` nếu document chưa active.

Frontend notes:

- Có thể dùng upload progress từ HTTP client.
- Không build R2 URL.
- Không hiển thị `storageKey` nếu response/detail/version có field này.

## Document Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| `GET` | `/documents/:id` | Yes | Chi tiết document kèm versions |
| `PUT` | `/documents/:id` | Yes | Cập nhật metadata document |

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| DELETE | /documents/:id | Yes | Soft delete document |
| POST | /documents/:id/versions | Yes | Upload version mới |
| GET | /documents/:id/versions | Yes | Lịch sử versions |
| GET | /documents/:id/download | Yes | Download current version |
| GET | /documents/:id/preview | Yes | Preview inline current version |
| GET | /documents/:id/versions/:versionId/preview | Yes | Preview version cụ thể |
| POST | /documents/:id/share | Yes | Tạo share link |
| GET | /share/documents/:token/preview | No | Preview qua share token |
| GET | /share/documents/:token/download | No | Download qua share token |

### GET /documents/:id

Response:

```
{
  "_id": "documentId",
  "name": "report.pdf",
  "description": "Quarter report",
  "status": "active",
  "currentVersionId": "versionId",
  "versions": [
    {
      "_id": "versionId",
      "documentId": "documentId",
      "versionNumber": 1,
      "originalName": "report.pdf",
      "mimeType": "application/pdf",
      "size": 12345,
      "uploadedBy": "userId",
      "createdAt": "2026-04-27T00:00:00.000Z"
    }
  ]
}
```

Security note:

- `DocumentVersion` model có `storageKey`. Nếu backend trả field này trong detail/versions, frontend không được hiển thị, log, hoặc dùng để build URL.

### PUT /documents/:id

Request:

```
{
  "name": "new-name.pdf",
  "description": "Updated description"
}
```

Response `200`: updated document object.

### `DELETE /documents/:id`

Response `204`. Backend set `status = "deleted"` và `deletedAt`.

### `POST /documents/:id/versions`

Content type: `multipart/form-data`.

Fields:

| Field | Type | Required |
|---|---|---|
| `file` | File | Yes |

Response `201`:

```
{
  "versionId": "versionId",
  "message": "Version uploaded"
}
```

Backend dùng `versionCounter` để reserve version number, tránh race khi upload song song.

### `GET /documents/:id/versions`

Response:

```
[
  {
    "_id": "versionId",
    "documentId": "documentId",
    "versionNumber": 2,
    "originalName": "report-v2.pdf",
    "mimeType": "application/pdf",
    "size": 23456,
    "scanStatus": "not_required",
    "createdAt": "2026-04-27T00:00:00.000Z"
  }
]
```

### Download/Preview

`GET /documents/:id/download` trả file stream attachment.

`GET /documents/:id/preview` trả file stream inline.

Headers quan trọng:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="report.pdf"; filename*=UTF-8''report.pdf
X-Content-Type-Options: nosniff
Cache-Control: private, no-store
```

Frontend nên:

- Gửi authenticated request.
- Nhận response dạng blob.
- Lấy filename từ `Content-Disposition` nếu có.
- Không cache download URL như public asset.

### `POST /documents/:id/share`

Request:

```
{
  "permission": "download",
  "mode": "fixed_version",
  "expiry": "2026-05-01T00:00:00.000Z",
  "maxDownloads": 5
}
```

Defaults:

- `permission`: `"view"`.
- `mode`: `"fixed_version"`.

Response `201`:

```
{
  "shareId": "shareId",
  "shareLink": "http://frontend/share/documents/share-token"
}
```

Share token raw chỉ xuất hiện trong `shareLink`. Backend lưu hash token.

## 5. Tasks

### Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| POST | /tasks | Yes | Tạo task |
| GET | /tasks | Yes | List task có filter |
| GET | /tasks/my | Yes | List task assign cho current user |
| GET | /projects/:projectId/tasks | Yes | List task theo projectId |
| GET | /departments/:departmentId/tasks | Yes | List task theo departmentId |
| GET | /tasks/:id | Yes | Chi tiết task kèm assignees/checklist/timeline |
| PATCH | /tasks/:id | Yes | Cập nhật task |
| DELETE | /tasks/:id | Yes | Soft delete task |
| POST | /tasks/:id/assignees | Yes | Thêm assignees |
| DELETE | /tasks/:id/assignees/:userId | Yes | Remove assignee |
| POST | /tasks/:id/checklist | Yes | Thêm checklist item |
| PATCH | /tasks/:id/checklist/:itemId | Yes | Cập nhật checklist item |
| DELETE | /tasks/:id/checklist/:itemId | Yes | Xóa checklist item |

### Frontend-facing Task Model

```json
{
  "_id": "taskId",
  "title": "Prepare report",
  "description": "",
  "projectId": null,
  "departmentId": null,
  "createdBy": "userId",
  "ownerId": "userId",
  "status": "todo",
  "priority": "medium",
  "startAt": null,
  "endAt": "2026-05-01T10:00:00.000Z",
  "completedAt": null,
  "archivedAt": null,
  "deletedAt": null,
  "createdAt": "2026-04-27T00:00:00.000Z",
  "updatedAt": "2026-04-27T00:00:00.000Z"
}
```

Frontend có thể map `_id` thành `id` trong UI state.

Status enum:

```
todo, in_progress, blocked, review, done, cancelled
```

Priority enum:

```
low, medium, high, urgent
```

### TaskAssignee Model

```
{
  "_id": "assignmentId",
  "taskId": "taskId",
  "userId": "userId",
  "assignedBy": "userId",
  "assignedAt": "2026-04-27T00:00:00.000Z",
  "removedAt": null,
  "status": "assigned"
}
```

Status enum:

```
assigned, accepted, declined, completed
```

### ChecklistItem Model

```
{
  "_id": "checklistItemId",
  "taskId": "taskId",
  "title": "Draft",
  "isDone": false,
  "order": 0,
  "createdBy": "userId",
  "completedBy": null,
  "completedAt": null,
  "createdAt": "2026-04-27T00:00:00.000Z",
  "updatedAt": "2026-04-27T00:00:00.000Z"
}
```

## `POST /tasks`

Request:

```json
{
  "title": "Prepare report",
  "description": "Quarter report",
  "projectId": null,
  "departmentId": null,
  "assigneeIds": ["userId"],
  "checklist": [
    {
      "title": "Draft",
      "isDone": false
    }
  ],
  "startAt": null,
  "endAt": "2026-05-01T10:00:00.000Z",
  "priority": "medium",
  "status": "todo"
}
```

Response `201`:

```json
{
  "_id": "taskId",
  "title": "Prepare report",
  "ownerId": "currentUserId",
  "createdBy": "currentUserId",
  "status": "todo",
  "priority": "medium",
  "assignees": [],
  "checklist": []
}
```

Permission notes:

- Admin có thể tạo task mọi scope.
- Non-admin hiện chỉ tạo standalone task (`projectId` và `departmentId` phải rỗng/null) vì chưa có membership model đáng tin cậy.
- Assignees phải là user tồn tại và active. User `locked`/`disabled` bị reject.

Error cases:

- `400`: thiếu title, assignee không hợp lệ/không tồn tại/inactive.
- `403`: không được tạo task trong scope.

### `GET /tasks`

Query params:

| Param | Ghi chú |
|---|---|
| `page` | default `1` |
| `size` | default `20`, max `100` |
| `status` | enum status |
| `priority` | enum priority |
| `projectId` | filter theo project |
| `departmentId` | filter theo department |
| `assigneeId` | filter theo assignee active |
| `createdBy` | filter theo creator |
| `dueBefore` | `endAt <= date` |
| `dueAfter` | `endAt >= date` |
| `search` | search title/description, backend escape regex và cắt 80 ký tự |

Response:

```
{
  "content": [
    {
      "_id": "taskId",
      "title": "Prepare report",
      "status": "todo",
      "priority": "medium"
    }
  ],
  "totalElements": 1
}
```

`totalElements` chỉ tính task current user đọc được.

Permission notes:

- Admin đọc tất cả.
- Creator/owner đọc được.
- Active assignee đọc được.
- Removed assignee không đọc được.
- Project/department membership hiện conservative vì chưa có model membership.

### `GET /tasks/my`

Response giống `/tasks`, nhưng backend set `assigneeId = currentUser`.

### `GET /projects/:projectId/tasks`

Tương đương `/tasks?projectId=:projectId`.

### `GET /departments/:departmentId/tasks`

Tương đương `/tasks?departmentId=:departmentId`.

### `GET /tasks/:id`

Response gồm task, assignees, checklist và activity timeline:

```
{
  "_id": "taskId",
  "title": "Prepare report",
  "status": "todo",
  "assignees": [
    {
      "_id": "assignmentId",
      "taskId": "taskId",
      "userId": "userId",
      "status": "assigned"
    }
  ],
  "checklist": [
    {
      "_id": "checklistItemId",
      "taskId": "taskId",
      "title": "Draft",
      "isDone": false
    }
  ],
  "timeline": [
    {
      "_id": "activityLogId",
      "action": "task.created",
      "entityType": "task",
      "entityId": "taskId",
      "metadata": {
        "title": "Prepare report"
      },
      "createdAt": "2026-04-27T00:00:00.000Z"
    }
  ]
}
```

Timeline lấy qua internal `listByEntity("task", taskId, { size: 10 })`; không có public endpoint task activities riêng.

### `PATCH /tasks/:id`

Allowed fields theo code:

```
{
  "title": "Updated title",
  "description": "Updated description",
  "status": "done",
  "priority": "high",
  "startAt": "2026-04-27T00:00:00.000Z",
  "endAt": "2026-05-01T10:00:00.000Z",
  "projectId": "projectId",
  "departmentId": "departmentId",
  "ownerId": "userId",
  "archivedAt": "2026-05-02T00:00:00.000Z"
}
```

Important implementation note:

- Code hiện tại vẫn nhận `ownerId`, `projectId`, `departmentId` trong PATCH nếu caller có quyền edit. Đây là điểm cần frontend-backend confirmation trước khi expose UI đổi owner/scope.
- Nếu `status` đổi sang `done`, backend set `completedAt`.
- Nếu `status` đổi khỏi `done`, backend set `completedAt = null`.

Permission:

- Admin hoặc creator/owner mới edit được.
- Active assignee chỉ đọc, không edit mặc định.

### `DELETE /tasks/:id`

Response `204`. Backend soft delete bằng `deletedAt`.

Permission:

- Admin hoặc creator/owner.

### `POST /tasks/:id/assignees`

Request:

```
{
  "userIds": ["userId1", "userId2"]
}
```

Hoặc:

```
{
  "userId": "userId"
}
```

Response:

```
{
  "assignees": [
    {
      "_id": "assignmentId",
      "taskId": "taskId",
      "userId": "userId",
      "assignedBy": "currentUserId",
      "status": "assigned",
      "removedAt": null
    }
  ]
}
```

Validation:

- Mỗi assignee ID phải là ObjectId hợp lệ.
- User phải tồn tại.
- User không được `locked`/`disabled`.
- Backend tránh duplicate bằng `findOneAndUpdate + upsert`.

Permission:

- Admin hoặc creator/owner.

### `DELETE /tasks/:id/assignees/:userId`

Response `204`. Backend set `removedAt`.

## Checklist Endpoints

### `POST /tasks/:id/checklist`

Request:

```
{
  "title": "Review",
  "isDone": false,
  "order": 1
}
```

Response `201`: checklist item.

### `PATCH /tasks/:id/checklist/:itemId`

Request:

```
{
  "title": "Review final",
  "isDone": true,
  "order": 2
}
```

Response `200`: updated checklist item.

Nếu `isDone` chuyển sang `true`, backend set `completedBy` và `completedAt`.

**`DELETE /tasks/:id/checklist/:itemId`**

Response `204`. Backend hard delete checklist item bằng `deleteOne`.

## 6. Notifications

### Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| `GET` | `/notifications` | Yes | List notification của current user |
| `GET` | `/notifications/unread-count` | Yes | Số notification chưa đọc |
| `PATCH` | `/notifications/:id/read` | Yes | Mark một notification đã đọc |
| `PATCH` | `/notifications/read-all` | Yes | Mark tất cả đã đọc |
| `DELETE` | `/notifications/:id` | Yes | Soft delete notification |
| `GET` | `/notifications/settings` | Yes | Lấy notification settings |
| `PATCH` | `/notifications/settings` | Yes | Cập nhật notification settings |

Notification luôn scoped theo current user. User không đọc/xóa được notification của user khác.

### Notification Model

```
{
  "_id": "notificationId",
  "userId": "userId",
  "type": "task_assigned",
  "title": "New task assigned",
  "message": "You were assigned to Prepare report",
  "entityType": "task",
  "entityId": "taskId",
  "actorId": "userId",
  "data": {
    "taskId": "taskId",
    "title": "Prepare report"
  },
```

```
    "readAt": null,
    "deletedAt": null,
    "createdAt": "2026-04-27T00:00:00.000Z"
  }
```

### GET /notifications

Query params:

| Param | Default | Ghi chú |
|---|---|---|
| page | 1 | |
| limit | 20 | max 100 |
| size | 20 | alternative cho limit |
| unreadOnly | - | "true" để chỉ lấy unread |
| type | - | ví dụ task_assigned |

Response:

```
  {
    "content": [],
    "totalElements": 0,
    "unreadCount": 0
  }
```

### GET /notifications/unread-count

Response:

```
  {
    "unreadCount": 3
  }
```

### PATCH /notifications/:id/read

Response 200: notification object đã có readAt.

### PATCH /notifications/read-all

Response:

```
  {
    "modifiedCount": 2
```

```
}
```

### `DELETE /notifications/:id`

Response `204`. Backend soft delete bằng `deletedAt`.

## NotificationSettings Model

```
{
  "_id": "settingsId",
  "userId": "userId",
  "inAppEnabled": true,
  "emailEnabled": false,
  "pushEnabled": false,
  "taskAssigned": true,
  "taskUpdated": true,
  "taskDueSoon": true,
  "documentShared": true,
  "documentVersionAdded": true,
  "adminActions": true,
  "createdAt": "2026-04-27T00:00:00.000Z",
  "updatedAt": "2026-04-27T00:00:00.000Z"
}
```

### `GET /notifications/settings`

Response `200`: settings object. Backend auto-create default settings nếu chưa có.

### `PATCH /notifications/settings`

Request:

```
{
  "inAppEnabled": true,
  "emailEnabled": false,
  "pushEnabled": false,
  "taskAssigned": true,
  "taskUpdated": true,
  "adminActions": true
}
```

Response `200`: updated settings.

Audit:

- Settings update tạo ActivityLog action `notification.settings_updated`.
- Read/delete notification không audit theo code hiện tại.

# 7. Admin and Audit

Admin endpoints yêu cầu:

```
Authorization: Bearer <admin-token>
```

User không phải admin nhận `403`.

## Endpoint Summary

| Method | Path | Auth | Mục đích |
|---|---|---|---|
| `GET` | `/admin/activity-logs` | Admin | List audit logs |
| `PATCH` | `/admin/users/:id/lock` | Admin | Khóa user |
| `PATCH` | `/admin/users/:id/unlock` | Admin | Mở khóa user |
| `POST` | `/admin/users/:id/reset-password` | Admin | Placeholder reset password |
| `GET` | `/admin/dashboard` | Admin | Dashboard stats |
| `GET` | `/admin/analytics/users` | Admin | User analytics |

## `GET /admin/activity-logs`

Supported filters:

| Param | Ghi chú |
|---|---|
| `actorId` | user thực hiện action |
| `actorType` | `user` hoặc `system` |
| `targetUserId` | user bị tác động |
| `entityType` | ví dụ `task`, `user`, `notification_settings` |
| `entityId` | id entity |
| `action` | action string |
| `projectId` | project id |
| `departmentId` | department id |
| `from` | map sang `createdAt >= from` |
| `to` | map sang `createdAt <= to` |
| `page` | default `1` |
| `limit` | default `20`, max `100` |
| `size` | alternative cho `limit` |

Response:

```
{
  "content": [
    {
      "_id": "activityLogId",
      "actorId": "userId",
      "actorType": "user",
      "action": "task.created",
      "entityType": "task",
      "entityId": "taskId",
      "targetUserId": null,
      "metadata": {
        "title": "Prepare report"
      },
      "createdAt": "2026-04-27T00:00:00.000Z"
    }
  ],
  "totalElements": 1
}
```

Invalid dates:

- `from`/`to` không parse được sẽ trả `400`.

Needs confirmation from implementation:

- `entityId` không được validate ObjectId thủ công; nếu invalid, Mongoose có thể trả cast error và đi qua error middleware.

### `PATCH /admin/users/:id/lock`

Request:

```
{
  "reason": "Policy violation"
}
```

Response `200`: user object đã sanitize password/OTP/reset fields.

Effects:

- Set `status = "locked"`.
- Set `lockedAt`, `lockedBy`, `lockReason`.
- Log `admin.user.locked`.
- Best-effort create notification type `admin_action`.

### `PATCH /admin/users/:id/unlock`

Response `200`: user object.

Effects:

- Set `status = "active"`.
- Clear lock fields.
- Log `admin.user.unlocked`.

### `POST /admin/users/:id/reset-password`

Current behavior:

- Log `admin.user.password_reset_requested`.
- Return `501`.
- Không tạo password plaintext.

Response:

```
{
  "message": "Admin password reset is not enabled yet. Use the existing forgot-
password flow."
}
```

### `GET /admin/dashboard`

Response:

```
{
  "totalUsers": 10,
  "activeUsers": 7,
  "lockedUsers": 2,
  "totalProjects": 0,
  "openTasks": 6,
  "completedTasks": 4,
  "overdueTasks": 3,
  "activeCalls": 0
}
```

`Project` và `Call` là optional models; nếu không có model, backend trả `0`.

### `GET /admin/analytics/users`

Query:

| Param | Required | Ghi chú |
|---|---|---|
| `granularity` | No | `day` hoặc `month`, default `day` |
| `from` | No | ISO date |

| Param | Required | Ghi chú |
|---|---|---|
| `to` | No | ISO date |

Response:

```
{
  "labels": ["2026-04-01", "2026-04-02"],
  "data": [2, 1]
}
```

## 8. ActivityLog

ActivityLog được dùng ở:

- Task: create/update/delete, add/remove assignees, add/update/complete/delete checklist.
- Notification settings: `notification.settings_updated`.
- Admin actions: lock/unlock/reset-password-requested.
- Document/folder events: hiện chưa thấy presenter document/folder gọi `logActivity`.

Frontend-facing fields:

```
{
  "_id": "activityLogId",
  "actorId": "userId",
  "actorType": "user",
  "action": "task.updated",
  "entityType": "task",
  "entityId": "taskId",
  "projectId": null,
  "departmentId": null,
  "targetUserId": null,
  "metadata": {
    "changedFields": ["status"]
  },
  "ipAddress": "127.0.0.1",
  "userAgent": "Mozilla/5.0",
  "createdAt": "2026-04-27T00:00:00.000Z"
}
```

Sensitive metadata redaction:

- Keys chứa `password`, `jwt`, `token`, `authorization`, `secret`, `apiKey`, `rawShareToken`, `fileContent`, v.v. sẽ bị thay bằng `[REDACTED]`.

Task detail timeline example:

```
{
  "action": "task.checklist_completed",
  "entityType": "task",
  "entityId": "taskId",
  "metadata": {
    "checklistItemId": "checklistItemId",
    "changedFields": ["isDone", "completedBy", "completedAt"]
  },
  "createdAt": "2026-04-27T00:00:00.000Z"
}
```

## 9. Permission Matrix

| Resource | Action | Ai có thể làm | Frontend behavior on `403` |
|---|---|---|---|
| Task | Read | Admin, creator, owner, active assignee | Ẩn detail hoặc show "Bạn không có quyền xem task này" |
| Task | Create standalone | Authenticated user | Show validation/permission message |
| Task | Create project/department scoped | Admin; non-admin hiện bị conservative nếu chưa có membership model | Hide scope selector hoặc handle `403` |
| Task | Edit | Admin, creator, owner | Disable edit controls nếu không owner/admin |
| Task | Assign/remove assignee | Admin, creator, owner | Disable assignee controls |
| Task | Delete | Admin, creator, owner | Disable delete action |
| Checklist | Add/update/delete | Theo task edit permission | Disable checklist editing |
| Notification | Read/list/delete/settings | Current user only | Nếu `404`, treat như không tồn tại/không thuộc user |
| Document | Upload | Admin/owner/creator/editor folder | Disable upload button |
| Document | Download/preview | User có read permission hoặc valid share token | Show permission message |
| Document | Share | User có edit permission | Disable share button |
| Admin routes | Any | `role = "admin"` | Redirect hoặc show "Admin only" |

Backend là source of truth. Frontend chỉ ẩn nút để UX tốt hơn, không được giả định permission ở client là đủ.

# 10. Frontend Integration Flows

## Create Task

1. User submit form title/description/priority/status/endAt.
2. Nếu là non-admin, ưu tiên tạo standalone task.
3. Gửi:

```
POST /api/tasks
Authorization: Bearer {{token}}
Content-Type: application/json
```

4. Nếu có assignees/checklist, có thể gửi cùng request.
5. Thành công: lưu `taskId`, refresh list/cache.
6. Xử lý:
    - `400`: thiếu title, assignee invalid/inactive.
    - `403`: không có quyền tạo trong scope.

## Task List With Filters

Gửi:

```
GET /api/tasks?status=todo&priority=medium&search=report&page=1&size=20
```

Render:

- `content`: danh sách task user đọc được.
- `totalElements`: số task đọc được, dùng cho pagination.

## Task Detail

Gửi:

```
GET /api/tasks/:id
```

Render:

- Task fields.
- Assignees.
- Checklist.
- Activity timeline.

## Notification Dropdown

1. Lấy unread count:

```
GET /api/notifications/unread-count
```

2. Lấy latest notifications:

```
GET /api/notifications?page=1&limit=10
```

3. Khi user click notification:

```
PATCH /api/notifications/:id/read
```

4. Nếu muốn clear all:

```
PATCH /api/notifications/read-all
```

### Upload Document

1. Chọn file từ input.
2. Validate sơ bộ extension/size ở frontend cho UX.
3. Gửi `multipart/form-data`:

```
POST /api/folders/:folderId/documents
field: file
field: description
```

4. Có thể gửi `Idempotency-Key` cho retry-safe UX.
5. Xử lý:
    - `400`: file type/content không hợp lệ.
    - `403`: không có quyền upload.
    - `413`: file quá lớn.

### Download Document

1. Gửi authenticated request:

```
GET /api/documents/:id/download
```

2. Nhận blob.
3. Parse filename từ `Content-Disposition`.
4. Không build R2 URL.

5. Không cache URL như public asset.

## 11. Not Supported / Known Limitations

- Project/department membership chưa được model đầy đủ. Task permission hiện conservative.
- Non-admin tạo project/department scoped task hiện có thể bị `403`.
- Realtime notification qua Socket.IO/Redis chưa được wire trong phase notifications.
- Email/push delivery chưa implement; settings được lưu trước.
- Notification read/delete không audit.
- Document/folder actions hiện chưa ghi ActivityLog.
- Virus scanning chưa implement; `scanStatus` tồn tại ở model version nhưng scanner không chạy.
- Admin reset password là placeholder `501`, không tạo password mới.
- `PATCH /tasks/:id` hiện code vẫn nhận `ownerId`, `projectId`, `departmentId` nếu caller có quyền edit. Frontend không nên expose UI đổi các field này cho tới khi backend chốt rule.
- Share link hiện là public token/fixed/latest mode; email-bound/password-protected chỉ là enum trong model, chưa thấy flow thực thi.
- Logout/refresh token endpoint chưa implement.
- `GET /tasks/:id/activities` chưa implement; timeline chỉ có trong `GET /tasks/:id`.

## 12. Frontend Checklist

- Luôn gửi `Authorization: Bearer {{token}}` cho protected endpoints.
- Gặp `401`: logout client hoặc đưa user về login.
- Gặp `403`: hiển thị permission/account status message.
- Không expose internal fields như `storageKey`, token hash, password/OTP/reset fields.
- Không cache document download URL như public asset.
- Không build R2 URL.
- Không gửi protected/sensitive fields trong PATCH task nếu UI không có rule rõ ràng.
- Validate frontend cho UX, nhưng backend validation là source of truth.
- Parse file download filename từ `Content-Disposition`.
- Với list có `totalElements`, hiểu đó là số resource user hiện tại có quyền đọc.
- Với admin UI, luôn xử lý `403` vì backend yêu cầu `role = "admin"`.